

Component Lifecycle Hooks

the events in a component's life, and how to catch them



Every Angular component has a **life**: it is born, it updates, and it is destroyed. Angular lets you run your own code at each of these moments using **lifecycle hooks**. Today we learn the main ones with tiny examples. Think of hooks as "call me when this happens."

1 What is a lifecycle hook?

A component goes through stages: Angular creates it, shows it, updates it when data changes, and finally removes it. At each stage, Angular can **call a method on your component**, if you have written one. That method is a lifecycle hook.

The idea in one line: a lifecycle hook is a method Angular runs automatically at a specific moment in the component's life. You write the method, Angular decides when to call it.

Each hook has a matching interface you can add for safety, and each method starts with `ng`.

2 The lifecycle, in order

Here are the main hooks, in the order Angular calls them:

Hook	When Angular calls it
<code>ngOnInit</code>	once, after the component is created and its inputs are set
<code>ngOnChanges</code>	whenever an <code>@Input</code> value changes
<code>ngDoCheck</code>	on every change detection run (used rarely)
<code>ngAfterViewInit</code>	once, after the component's view is fully ready
<code>ngOnDestroy</code>	once, just before the component is removed

You do not need all of them. Most of the time you will use just two: `ngOnInit` to set things up, and `ngOnDestroy` to clean things up. Start with those.

3 ngOnInit: the most used hook

`ngOnInit` runs **once**, right after the component is created. This is the place to load data, set starting values, or call an API. It is the "the component is ready, do your setup now" moment.

HELLO.COMPONENT.TS

```
import { Component, OnInit } from "@angular/core";

@Component({
  selector: "app-hello",
  template: `<h3>{{ message }}</h3>`,
})
export class HelloComponent implements OnInit {
  message: string = "";

  ngOnInit() {
    this.message = "Component is ready!";
    console.log("ngOnInit ran");
  }
}
```

When the component appears, `ngOnInit` runs, sets the message, and the page shows **Component is ready!**. This is where your setup code belongs, not in the constructor.

Common question: why not use the constructor? The constructor runs before Angular has set up the component's inputs and view. `ngOnInit` runs after, so your data is ready. Rule of thumb: constructor for simple wiring, `ngOnInit` for real setup work.

4 `ngOnChanges`: react to input changes

`ngOnChanges` runs whenever a parent changes an `@Input` value. Use it when the child needs to react to new data coming in.

CHILD.COMPONENT.TS

```
import { Component, Input, OnChanges } from "@angular/core";

@Component({
  selector: "app-child",
  template: `<p>Name is: {{ name }}</p>`,
})
export class ChildComponent implements OnChanges {
  @Input() name: string = "";

  ngOnChanges() {
    console.log("Input changed, name is now:", this.name);
  }
}
```

Every time the parent sets a new `name`, `ngOnChanges` runs and you can respond to the new value.

5 `ngOnDestroy`: clean up before leaving

`ngOnDestroy` runs **once**, just before Angular removes the component. This is where you clean up: stop timers, cancel subscriptions, remove listeners. Skipping cleanup causes memory leaks.

TIMER.COMPONENT.TS

```
import { Component, OnInit, OnDestroy } from "@angular/core";

@Component({
  selector: "app-timer",
  template: `<p>Seconds: {{ count }}</p>`,
})
export class TimerComponent implements OnInit, OnDestroy {
  count: number = 0;
  timerId: any;

  ngOnInit() {
    // start a timer when the component appears
    this.timerId = setInterval(() => this.count++, 1000);
  }

  ngOnDestroy() {
    // stop the timer before the component is removed
    clearInterval(this.timerId);
    console.log("Timer cleaned up");
  }
}
```

The timer starts in `ngOnInit` and is stopped in `ngOnDestroy`. Without the cleanup, the timer would keep running even after the component is gone, wasting memory. Start in init, clean up in destroy.

6 A simple way to remember the order

Born: constructor runs, then `ngOnChanges` (first inputs), then `ngOnInit` (setup).

Living: `ngOnChanges` runs again each time an input changes.

View ready: `ngAfterViewInit` runs once the template is fully rendered.

Dying: `ngOnDestroy` runs just before the component is removed.

7 Remember this

Lifecycle hooks are methods Angular calls automatically at set moments in a component's life.

ngOnInit – runs once at the start. Do your setup here (load data, set values).

ngOnChanges – runs when an `@Input` changes. React to new data.

ngOnDestroy – runs once at the end. Clean up here (timers, subscriptions).

Rule of thumb: set up in `ngOnInit`, clean up in `ngOnDestroy`. Those two cover most of your needs.

8 Homework

Add `ngOnInit` to a component and `console.log` a message from it. Watch it print once when the page loads.

Build the timer example: start a counter in `ngOnInit`, stop it in `ngOnDestroy`. Confirm cleanup runs by logging inside it.

Try `ngOnChanges`: pass a value from parent to child with `@Input`, change it, and log inside `ngOnChanges`.

In one line each: what do `ngOnInit` and `ngOnDestroy` do, and when does each run?